

A-QUIZE

Real-Time Online Quiz System

Project Report

Technology: ASP.NET Core 8 | SignalR | MySQL | Entity Framework Core
June 2026

1. Problem Statement

Traditional quiz and examination systems are typically static, offline, and unable to provide real-time engagement between hosts and participants. Educators and event organizers often struggle with:

- Lack of real-time feedback — participants must wait for manual grading after the quiz ends.
- No live visibility — hosts cannot monitor who has joined or how scores are progressing.
- Inconvenient setup — paper-based or file-based quizzes require significant manual work.
- Poor engagement — static formats provide no countdown, navigation flexibility, or live leaderboard.

- No instant scoring — participants do not receive their results immediately upon submission.

A-Quizze addresses all of these problems by delivering a browser-based, real-time quiz platform that allows a host to create and publish a quiz, and participants to join, attempt, and receive results instantly — with live leaderboard updates for everyone in the session.

2. System Design

2.1 Architecture Overview

The application follows an N-tier web architecture built on ASP.NET Core Razor Pages. The key layers are:

- Presentation Layer: Razor Pages (.cshtml) for server-rendered UI, with JavaScript for client-side interactivity.
- Application Layer: Page model classes (.cshtml.cs) containing business logic and request handling.
- Real-Time Layer: SignalR Hub (QuizHub) that pushes live events to connected clients via WebSocket.
- Data Layer: Entity Framework Core with a MySQL database, using code-first schema generation.

2.2 Database Schema

The database consists of five core entities:

Entity	Responsibility
QuizSession	Stores quiz title, description, 6-character code, question count, total time, and publication/start/finish state.
Question	Stores question text, four answer options (JSON), correct option index, points, and display order.
AppUser	Student accounts with name, email, and plaintext password (starter project).
Participant	Links a user to a quiz session; tracks start time, end time, score, and submission state.
ParticipantAnswer	Records each answer selected by a participant with correctness and points awarded.

2.3 Real-Time Communication (SignalR)

SignalR is used for two key real-time flows:

- **Published Quiz Updates:** Students subscribe to a 'published-quizzes' SignalR group. When the host publishes a quiz, the server broadcasts a `PublishedQuizzesChanged` event, and students' quiz lists refresh automatically without a page reload.
- **Live Leaderboard:** When any participant submits their quiz, the server recalculates the leaderboard and broadcasts a `LeaderboardUpdated` event to all members of that quiz session's group — both the host room and participants.

2.4 Quiz Flow

The end-to-end flow of the system is:

Step	Description
1. Create	Host logs in and creates a quiz with a title, description, planned question count, and total time limit. A unique 6-character code is auto-generated.
2. Build	Host navigates to <code>BuildQuiz</code> and adds questions one by one, each with four options, a correct answer, and point value.
3. Publish	After all questions are added, the host publishes the quiz. Students are notified in real time.
4. Join	Students register/login, see the published quiz in the Quizzes list, and click <code>Start Quiz</code> .
5. Play	Students navigate questions with <code>Previous/Next</code> controls, select answers, and submit. A countdown timer runs for the whole attempt.
6. Score	On submission, the server auto-scores the attempt and broadcasts the updated leaderboard.
7. Result	The participant sees their score. The host and all participants see the live leaderboard.

3. Tools & Technologies Used

Tool / Technology	Purpose
ASP.NET Core 8 (Razor Pages)	Server-side web framework for page rendering and HTTP request handling.
SignalR	Real-time WebSocket communication for live leaderboard and quiz list updates.
Entity Framework Core	ORM for database access and code-first schema creation.

MySQL / Pomelo.EntityFrameworkCore	Relational database for storing all quiz, user, and participant data.
C# 12	Primary programming language for all back-end logic.
HTML / CSS / JavaScript	Front-end presentation, countdown timer, answer selection, and auto-submit logic.
ASP.NET Core Session	Server-side session for host and student authentication state.
System.Text.Json	Serialization of question options and submitted answers.
Git / GitHub	Version control and source code management.

4. Features Implemented

4.1 Host Features

- Secure host login via a configurable admin password in appsettings.json.
- Create a quiz with title, description, planned question count, and total time limit.
- Add questions one by one with four answer options, selectable correct answer, and custom point value.
- Publish a quiz to make it visible to students — triggers a live SignalR notification.
- View a host room dashboard for the active quiz.
- See the live leaderboard update in real time as students submit.

4.2 Student Features

- Student registration with name and email.
- Student login to access published quizzes.
- View all available published quizzes with live refresh when new ones are published.
- Start a quiz attempt with a per-participant overall countdown timer.
- Navigate between questions using Previous and Next buttons.
- Change selected answers at any time before submitting.
- Manual submission or automatic submission when the countdown expires.
- Immediate score display upon submission.
- View the live leaderboard after submission.

4.3 System Features

- Unique 6-character alphanumeric quiz codes generated for each session.
- Auto-scoring on submission: each answer is checked against the stored correct option index.
- Real-time leaderboard sorted by score descending, then by submission time ascending.

- Database schema auto-created on startup via `EnsureCreated()`.
- Session-based authentication for both host and students (4-hour idle timeout).
- Duplicate participant prevention: a user can only have one attempt per quiz session.

5. Conclusion

A-Quiz demonstrates a fully functional real-time quiz platform built with modern .NET web technologies. The combination of ASP.NET Core Razor Pages for server-rendered pages, SignalR for live updates, and Entity Framework Core for database persistence produces a clean and maintainable architecture.

The system covers the complete quiz lifecycle from creation and question building, through publication and participant play, to auto-scoring and live leaderboard display — all within a single web application with no external quiz service dependency.

As noted in the project README, this is a starter project. Recommended improvements for a production deployment include password hashing (e.g., ASP.NET Core Identity or BCrypt), role-based authorization middleware, HTTPS enforcement, input validation hardening, and scalable SignalR backplane support (e.g., Azure SignalR Service) for multiple server instances.